Deep Learning Oral Exam Preparation

Part 1: Practical 2 — Feedforward Neural Network (FNN)

1. Aim of the practical

To implement a Feedforward Neural Network using deep learning to classify handwritten digits from the MNIST dataset.

2. What is a Feedforward Neural Network?

A Feedforward Neural Network (FNN) is an artificial neural network in which information moves in only one direction: from the input layer to the hidden layers and then to the output layer.

There are no loops or feedback connections.

Example: Handwritten digit recognition.

- Input: An image of a handwritten digit.
- Processing: The neural network learns patterns in the image.
- Output: The predicted digit, such as 0, 1, 2, or 9.

3. What is the MNIST dataset?

MNIST is a dataset of handwritten digit images.

- It contains 70,000 grayscale images.
- 60,000 images are used for training.
- 10,000 images are used for testing.
- Each image has 28 × 28 pixels.
- There are 10 output classes, representing digits 0–9.

4. Explain the working of the FNN practical

Input image

28 × 28 grayscale pixels

Flatten layer

Converts 28 × 28 into 784 values

Dense hidden layer + ReLU

Learns patterns from input values

Output layer + Softmax

Predicts probabilities for digits 0–9

Steps:

1. Import the required libraries.
2. Load the MNIST dataset.
3. Normalize pixel values from 0–255 to 0–1.
4. Build the neural network using Keras.
5. Compile the model with an optimizer, loss function, and metric.
6. Train the model using the training images.
7. Evaluate its performance on test images.
8. Predict the digit for an input image.

5. Important functions and their uses

Function	Use
mnist.load_data()	Loads the MNIST dataset
Flatten()	Converts a 2D image into a 1D vector
Dense()	Creates a fully connected neural-network layer
relu	Adds non-linearity to the hidden layer
softmax	Produces probabilities for output classes
model.compile()	Configures training
model.fit()	Trains the model
model.evaluate()	Evaluates the model
model.predict()	Generates predictions

6. Expected viva questions and answers

Q1. What is deep learning?

Deep learning is a branch of machine learning that uses neural networks with multiple layers to learn patterns from data.

Q2. What is an epoch?

One epoch means the model has processed the entire training dataset once.

Q3. What is a batch size?

Batch size is the number of training samples processed before the model updates its weights.

Q4. What is an activation function?

An activation function determines the output of a neuron and introduces non-linearity into a neural network.

Q5. What is ReLU?

ReLU stands for Rectified Linear Unit. It returns zero for negative inputs and the input itself for positive inputs.

$$ f(x)=\max(0,x) $$

Q6. Why is Softmax used in the output layer?

Softmax converts output scores into probabilities. It is commonly used for multiclass classification.

Q7. What is normalization?

Normalization scales data to a suitable range. In MNIST, dividing pixel values by 255 converts them from 0–255 to 0–1.

Q8. What is a loss function?

A loss function measures the difference between the model's predictions and the actual answers.

Q9. What is an optimizer?

An optimizer updates the model's weights to reduce the loss.

Q10. What is backpropagation?

Backpropagation calculates how much each weight contributes to the prediction error and computes gradients used to update the weights.

Part 2: Practical 3 — Image Classification Using CNN

1. Aim

To implement a Convolutional Neural Network (CNN) for image classification.

2. What is a CNN?

A Convolutional Neural Network is a deep learning model designed mainly for image and other grid-like data.

It automatically learns features such as edges, shapes, textures, and more complex patterns.

CNNs are widely used in:

- Image classification
- Face recognition

- Medical image analysis
- Object detection
- Image segmentation

3. CNN architecture

Input image

Convolution layerExtracts features

ReLU activationIntroduces non-linearity

Max poolingReduces spatial dimensions

Flatten + Dense

Output prediction

4. Important CNN functions

Function	Explanation
Conv2D()	Applies filters to extract image features
MaxPooling2D()	Reduces the feature map size
Flatten()	Converts feature maps into a vector
Dense()	Connects neurons fully between layers
model.fit()	Trains the model
model.evaluate()	Evaluates performance
reshape()	Changes the dimensions of an array

5. Expected viva questions

Q1. What is convolution?

Convolution is an operation in which a small filter slides over an image to detect features such as edges, lines, and textures.

Q2. What is a filter or kernel?

A filter is a small matrix of learnable values used to extract features from an image.

Q3. What is a feature map?

A feature map is the output produced when a convolution filter processes an image.

Q4. What is pooling?

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Q5. What is Max Pooling?

Max pooling selects the maximum value from each local region of a feature map.

Q6. Why is CNN better than a basic FNN for image data?

CNNs preserve spatial relationships and learn local image features using filters and shared weights. A basic FNN generally flattens the image first.

Q7. What is parameter sharing?

Parameter sharing means the same convolution filter weights are used at different spatial locations in an image.

Q8. What is translation equivariance?

Translation equivariance means that shifting an input generally shifts the corresponding feature representation in a related way.

Q9. What is AlexNet?

AlexNet is a deep CNN architecture that achieved a major breakthrough in large-scale image classification in 2012.

Q10. What is the difference between FNN and CNN?

An FNN commonly uses fully connected layers, whereas a CNN uses convolution filters to learn spatial features efficiently.

Part 3: Practical 4 — Autoencoder for Anomaly Detection

1. Aim

To implement an autoencoder for anomaly detection using reconstruction error.

Important: Your practical code uses creditcard.csv and separates normal transactions from fraud transactions. Explain it as credit-card fraud/anomaly detection if that is the code your examiner asks about.

2. What is an autoencoder?

An autoencoder is a neural network that learns to reconstruct its input.

It has three main parts:

1. Encoder: Compresses the input into a smaller representation.
2. Latent space: Stores the compressed representation.
3. Decoder: Reconstructs the original input from that representation.

Input

Encoder

Latent

Compressed representation is decoded

Reconstructed input

Decoder

3. What is reconstruction error?

Reconstruction error measures the difference between the original input and the reconstructed output.

A common measure is Mean Squared Error (MSE):

\[MSE=\frac{1}{n}\sum_{i=1}^{n}(x_i-\hat{x}_i)^2 \]

Here:

- \(x_i\) is the original value.
- \(\hat{x}_i\) is the reconstructed value.
- \(n\) is the number of input features.

If a transaction has a high reconstruction error, it may be anomalous.

4. Working of the practical

1. Load the transaction dataset.
2. Separate normal and fraudulent transactions using the labels.
3. Train the autoencoder primarily on normal transactions.
4. Reconstruct the input transactions.
5. Calculate reconstruction error.
6. Choose a threshold for identifying anomalies.
7. Flag transactions with errors above the threshold.

5. Expected viva questions

Q1. Why is an autoencoder used for anomaly detection?

An autoencoder learns the patterns of normal data. Unusual data may not be reconstructed accurately, resulting in a higher reconstruction error.

Q2. What is the encoder?

The encoder converts the input into a compressed representation.

Q3. What is the decoder?

The decoder reconstructs the original input from the compressed representation.

Q4. What is latent space?

Latent space is the compressed representation learned by the encoder.

Q5. What is an anomaly?

An anomaly is a data point that differs significantly from normal patterns.

Q6. Why train on normal data?

Training mainly on normal data helps the model learn normal patterns, making unusual examples easier to identify through reconstruction error.

Q7. What is a threshold?

A threshold is the selected error value used to separate normal-looking samples from suspected anomalies.

Q8. What happens if the threshold is too low?

Too many normal transactions may be flagged as anomalies, increasing false positives.

Q9. What happens if the threshold is too high?

Some genuine fraudulent transactions may be missed, increasing false negatives.

Q10. Is every high-error transaction definitely fraud?

No. It is only a suspected anomaly. The threshold and model performance must be evaluated.

Part 4: Practical 5 — CBOW (Continuous Bag of Words)

1. Aim

To implement the Continuous Bag of Words model to predict a target word from its surrounding context words.

2. What is NLP?

Natural Language Processing (NLP) is a field of artificial intelligence that enables computers to process and understand human language.

Examples include:

- Chatbots
- Translation
- Sentiment analysis
- Text summarization
- Search engines

3. What is CBOW?

CBOW is a Word2Vec architecture that predicts a target word using the surrounding context words.

Example sentence:

Deep learning is very useful.

If the target word is learning, the context may include Deep and is.

The model learns to predict the missing target word from the context.

4. CBOW working

Context words

Words surrounding the target

Word embeddings

Represent words as numerical vectors

Output layer

Assigns scores/probabilities to vocabulary words

Predicted target word

5. Important concepts

- Tokenization: Splitting text into individual words or tokens.
- Vocabulary: The set of unique words used by the model.
- Word index: A numerical identifier assigned to each vocabulary word.
- Embedding: A dense numerical vector representing a word.
- Context window: The number of nearby words used to predict a target.
- Loss: Measures how far the predicted target distribution is from the correct target.

6. Expected viva questions

Q1. What is CBOW?

CBOW is a Word2Vec model that predicts a target word from its surrounding context words.

Q2. What is the difference between CBOW and Skip-gram?

CBOW predicts the target word from context words, while Skip-gram predicts surrounding context words from a target word.

Q3. What is an embedding?

An embedding represents a word as a dense numerical vector that can capture useful relationships between words.

Q4. Why is tokenization necessary?

Tokenization converts text into units that can be processed by a computer.

Q5. What is a vocabulary?

Vocabulary is the collection of unique words recognized by the model.

Q6. Why are words converted to numbers?

Neural networks operate on numerical data, so words must be represented numerically.

Q7. What is a context window?

A context window determines how many surrounding words are used to predict the target word.

Q8. What is Softmax used for in CBOW?

Softmax converts output scores into a probability distribution over the vocabulary.

Q9. What is an embedding layer?

An embedding layer maps integer word indices to trainable dense vectors.

Q10. What is the purpose of the loss function?

It measures the prediction error and guides the model's learning process.

Part 5: Practical 6 — Image Classification Using VGG16

1. Aim

To use transfer learning with a pretrained VGG16 network for image classification.

Important viva correction: Although your practical PDF calls this object detection, the described VGG16 code performs image classification. Object detection usually also identifies object locations using bounding boxes.

2. What is VGG16?

VGG16 is a convolutional neural network architecture with 16 weight-bearing layers. It is commonly used for image classification and feature extraction.

3. What is transfer learning?

Transfer learning means reusing knowledge learned by a model on one dataset for a different task.

For example, a pretrained image model may already recognize edges, textures, shapes, and other visual patterns. We can reuse these features instead of training an entire network from scratch.

4. Working of the practical

1. Load the image dataset from folders.
2. Resize images to the required dimensions.
3. Rescale pixel values.
4. Load pretrained VGG16 without its original classification head.
5. Freeze the pretrained layers.
6. Add new classification layers.
7. Compile and train the new classifier.
8. Evaluate the model on unseen images.

5. Important functions

Function	Purpose
ImageDataGenerator()	Prepares and preprocesses image batches
flow_from_directory()	Loads images from class-specific folders
VGG16()	Loads the pretrained VGG16 architecture
include_top=False	Removes VGG16's original final classification layers
trainable=False	Freezes a layer so its weights are not updated during training
Dropout()	Reduces overfitting by randomly disabling some units during training
Adam	Optimizer used to update trainable weights
categorical_crossentropy	Loss function commonly used for multiclass one-hot labels

6. Expected viva questions

Q1. What is VGG16?

VGG16 is a CNN architecture used for image classification and feature extraction.

Q2. What is transfer learning?

Transfer learning reuses a pretrained model's learned features for a new task.

Q3. What does include_top=False mean?

It excludes the original final classification layers of VGG16 so that new task-specific layers can be added.

Q4. Why freeze pretrained layers?

Freezing layers preserves their learned features and reduces the number of trainable parameters.

Q5. What is fine-tuning?

Fine-tuning means unfreezing some pretrained layers and training them carefully on the new dataset.

Q6. What is Dropout?

Dropout randomly disables a portion of units during training to help reduce overfitting.

Q7. What is overfitting?

Overfitting occurs when a model learns the training data too closely and performs poorly on new data.

Q8. What is the difference between image classification and object detection?

Image classification predicts the class of an image. Object detection identifies objects and their locations, typically with bounding boxes.

Q9. Why use a pretrained model?

It can reduce training time and data requirements by reusing features learned from a large dataset.

Q10. What is the difference between feature extraction and fine-tuning?

Feature extraction keeps the pretrained layers fixed and trains new layers. Fine-tuning also trains selected pretrained layers.

Part 6: Theory Viva — Unit I: Fundamentals of Deep Learning

1. What is an Artificial Neural Network (ANN)?

An ANN is a computing model inspired by the way biological neurons process information. It contains interconnected artificial neurons arranged in layers.

2. What is a Multilayer Perceptron (MLP)?

An MLP is a feedforward neural network containing an input layer, one or more hidden layers, and an output layer. It commonly uses fully connected layers and nonlinear activation functions.

3. What is a feedforward network?

A feedforward network passes information from input to output without recurrent loops.

4. What is backpropagation?

Backpropagation calculates gradients of the loss with respect to model parameters. These gradients are then used by an optimizer to update the parameters.

5. What is gradient descent?

Gradient descent is an optimization method that adjusts model parameters in the direction that reduces the loss.

$$\theta_{\text{new}}=\theta_{\text{old}}-\eta\nabla_\theta L$$

Here, η is the learning rate and L is the loss.

6. What is a learning rate?

The learning rate controls the size of the weight updates.

- Too high: Training may become unstable.
- Too low: Training may take a long time.

7. What is the vanishing gradient problem?

It occurs when gradients become extremely small as they propagate backward through many layers. Earlier layers then learn very slowly.

8. What is ReLU?

ReLU is an activation function defined as:

$$f(x)=\max(0,x)$$

It is simple and widely used in hidden layers.

9. What are Leaky ReLU and ELU?

- Leaky ReLU: Allows a small negative output for negative inputs.
- ELU: Uses a smooth exponential function for negative inputs and behaves linearly for positive inputs.

Both can help address some limitations associated with ordinary ReLU.

10. What is regularization?

Regularization refers to techniques that reduce overfitting and help a model generalize to unseen data.

Examples include L1 regularization, L2 regularization, and dropout.

11. What is a hyperparameter?

A hyperparameter is a setting chosen before or during model configuration rather than directly learned as an ordinary model weight.

Examples include learning rate, batch size, number of epochs, and number of hidden layers.

Part 7: Theory Viva — Unit II: Convolutional Neural Networks

1. What is a CNN?

A CNN is a neural network that uses convolution operations to learn spatial features from images.

2. What is a convolution layer?

It applies learnable filters to input data to generate feature maps.

3. What is parameter sharing?

The same filter weights are reused at multiple image positions, reducing the number of parameters.

4. What is pooling?

Pooling reduces the spatial size of feature maps.

5. What is the difference between Max Pooling and Average Pooling?

Max Pooling	Average Pooling
Selects the maximum value	Calculates the average
Retains strong local activations	Retains average local information
Commonly used for prominent features	Useful for summarizing regions

6. What is AlexNet?

AlexNet is a CNN architecture that became famous after its success in the ImageNet image-classification competition in 2012. It helped demonstrate the effectiveness of deep CNNs for large-scale vision tasks.

7. What is stride?

Stride is the number of pixels a convolution filter moves each time it slides across an image.

8. What is padding?

Padding adds pixels, usually zeros, around the input's boundary. It can help control the output dimensions.

9. What is a feature map?

A feature map is the output generated by applying a filter to an input.

10. What is the role of ReLU in CNNs?

ReLU introduces non-linearity so the network can learn more complex patterns.

Part 8: Theory Viva — Unit III: Recurrent Neural Networks

1. What is an RNN?

A Recurrent Neural Network is a neural network designed to process sequential data. It maintains a hidden state that carries information from earlier steps.

Examples include text, speech, and time-series data.

2. What is the difference between FNN and RNN?

FNN	RNN
Usually processes fixed-shaped inputs	Designed for sequential inputs
Has no recurrent connections	Uses recurrent connections
Does not inherently maintain a sequence state	Maintains a hidden state
Useful for ordinary classification tasks	Useful for sequence-related tasks

3. What is a hidden state?

A hidden state is an internal representation that carries information from earlier sequence steps to later ones.

4. What is LSTM?

Long Short-Term Memory (LSTM) is a type of RNN designed to learn long-range dependencies more effectively. It uses a cell state and gates to regulate information flow.

5. What are the gates in LSTM?

- Forget gate: Decides what information to discard.
- Input gate: Decides what new information to store.
- Output gate: Controls what information is exposed as the hidden output.

6. What is an encoder-decoder model?

An encoder-decoder model converts an input sequence into a representation and uses a decoder to generate an output sequence.

Example: Machine translation.

7. What is a recursive neural network?

A recursive neural network applies neural computations over hierarchical structures, such as parse trees. It is different from a recurrent network, which processes sequences step by step.

8. What is the vanishing gradient problem in RNNs?

During training over long sequences, gradients can become very small, making it difficult to learn long-term dependencies.

9. Where are RNNs used?

RNNs are used in sequence modelling, language processing, speech applications, and time-series prediction.

Part 9: Theory Viva — Unit IV: Autoencoders

1. What is an autoencoder?

An autoencoder learns to reconstruct input data through a compressed or constrained representation.

2. What is an undercomplete autoencoder?

An undercomplete autoencoder has a latent representation with fewer dimensions than the input, encouraging the model to compress the information.

3. What is a sparse autoencoder?

A sparse autoencoder encourages only a small number of latent units to activate for a given input.

4. What is a denoising autoencoder?

A denoising autoencoder learns to reconstruct clean data from a corrupted or noisy input.

5. What is a contractive autoencoder?

A contractive autoencoder uses a penalty that encourages the learned representation to be less sensitive to small changes in the input.

6. What is a stochastic autoencoder?

The term can refer to autoencoder approaches that introduce randomness into the representation or encoding process. The exact meaning depends on the model described in the course material.

7. What are the applications of autoencoders?

- Data compression
- Feature learning
- Noise removal
- Anomaly detection
- Dimensionality reduction

8. What is the difference between an autoencoder and PCA?

PCA is a linear dimensionality-reduction technique. Autoencoders can learn nonlinear representations when built with nonlinear layers.

9. Why is reconstruction loss important?

It measures how closely the reconstructed output matches the input and provides a learning objective for the autoencoder.

Part 10: Theory Viva — Unit V: Representation Learning

1. What is representation learning?

Representation learning allows a model to learn useful features or representations from raw data automatically.

2. What is greedy layer-wise pretraining?

It is a training approach in which layers or groups of layers are pretrained one at a time before the whole network is trained or fine-tuned together.

3. What is transfer learning?

Transfer learning reuses knowledge from a model trained on one task for another related task.

4. What is domain adaptation?

Domain adaptation aims to adapt a model trained on one data distribution or domain so that it performs well on another domain.

Example: Adapting an image classifier trained on clear images to images captured in different lighting conditions.

5. What is a distributed representation?

A distributed representation uses a pattern of activations across multiple units to represent an input or concept.

6. What is DenseNet?

DenseNet stands for Densely Connected Convolutional Network. Each layer receives feature maps from earlier layers within a dense block, encouraging feature reuse and improving information flow.

7. What are the advantages of DenseNet?

- Feature reuse
- Improved information flow
- Potentially fewer parameters than some comparable architectures
- Better gradient propagation through dense connections

8. What is the difference between transfer learning and domain adaptation?

Transfer learning broadly reuses knowledge from a source task. Domain adaptation specifically focuses on adapting to differences between source and target data domains.

Part 11: Theory Viva — Unit VI: Applications of Deep Learning

1. What is image classification?

Image classification assigns a class label to an image.

Example: Identifying whether an image contains a cat or a dog.

2. What is social network analysis?

Social network analysis studies relationships and interactions among people, groups, or accounts. Deep learning can help identify communities, predict links, and detect unusual patterns.

3. How is deep learning used in speech recognition?

Deep learning models process audio features and learn to map spoken sounds to words or text.

4. What is a recommender system?

A recommender system suggests items that a user may find useful or interesting.

Examples include movie, product, and music recommendations.

5. How is deep learning used in NLP?

Deep learning is used for translation, sentiment analysis, summarization, question answering, text generation, and language understanding.

6. What is the difference between classification and detection?

Classification predicts a class label. Object detection predicts classes as well as the locations of objects in an image.

7. What is speech-to-text?

Speech-to-text converts spoken audio into written text using speech recognition.

8. What is sentiment analysis?

Sentiment analysis identifies the sentiment expressed in text, such as positive, negative, or neutral.

Part 12: Important Differences for Quick Revision

Topic	First concept	Second concept
FNN vs RNN	No recurrent state	Maintains a sequence-related hidden state
FNN vs CNN	Often fully connected	Uses convolution filters
ReLU vs Softmax	Common hidden-layer activation	Common multiclass output activation

Max vs Average Pooling	Takes maximum	Takes average
CBOW vs Skip-gram	Context predicts target	Target predicts context
Classification vs Detection	Predicts image class	Predicts object classes and locations
Feature extraction vs Fine-tuning	Pretrained layers remain frozen	Selected pretrained layers are updated
Undercomplete vs Sparse Autoencoder	Uses a lower-dimensional latent representation	Encourages sparse activations
Training vs Testing	Learns model parameters	Evaluates generalization

Part 13: Common Library Questions

Library or module	**Purpose**
TensorFlow	Deep learning and numerical computation
Keras	High-level neural-network API
NumPy	Numerical arrays and mathematical operations
Pandas	Tabular data processing
Matplotlib	Plotting graphs
Seaborn	Statistical visualizations
scikit-learn	Machine learning utilities and evaluation tools
random	Random-number generation
re	Regular expressions and text cleaning
ImageDataGenerator	Image preprocessing and augmentation utilities
VGG16	Pretrained CNN architecture
SGD	Stochastic Gradient Descent optimizer

Common viva questions

Q1. What is TensorFlow?

TensorFlow is a framework for numerical computation and machine learning, including deep learning.

Q2. What is Keras?

Keras provides a high-level API for building, training, and evaluating neural networks.

Q3. What is NumPy?

NumPy is a Python library for efficient numerical arrays and mathematical operations.

Q4. What is Pandas?

Pandas is used for loading, cleaning, transforming, and analyzing structured data.

Q5. What is Matplotlib?

Matplotlib is used to create graphs such as line plots, bar charts, and loss curves.

Q6. What is scikit-learn?

Scikit-learn provides machine learning utilities for preprocessing, model selection, and evaluation.

Q7. What is an optimizer?

An optimizer updates model parameters to reduce the loss.

Q8. What is an epoch?

An epoch is one complete pass through the training dataset.

Q9. What is model accuracy?

Accuracy is the fraction of predictions that are correct.

$$ Accuracy=\frac{\text{Correct Predictions}}{\text{Total Predictions}} $$

Q10. What is a loss curve?

A loss curve plots the loss value across training epochs to help monitor the learning process.

Final 10-minute oral exam revision

Memorize these short answers first.

1. FNN: A neural network in which information flows from input to output without recurrent loops.
2. CNN: A neural network that learns image features using convolution filters.
3. RNN: A neural network designed to process sequences using recurrent connections and hidden states.
4. LSTM: A type of RNN that uses gates and a cell state to learn long-term dependencies.
5. Autoencoder: A network that learns to reconstruct its input.
6. Reconstruction error: A measure of the difference between original and reconstructed data.

7. CBOW: Predicts a target word using surrounding context words.
8. VGG16: A CNN architecture commonly used for image classification and feature extraction.
9. Transfer learning: Reusing knowledge from a pretrained model for a new task.
10. Backpropagation: Computes gradients used to update neural-network weights.
11. Gradient descent: An optimization method that updates parameters to reduce loss.
12. ReLU: Returns zero for negative inputs and the input itself for positive inputs.
13. Softmax: Converts output scores into probabilities.
14. Pooling: Reduces feature-map dimensions.
15. Dropout: Randomly disables some units during training to help prevent overfitting.
16. Overfitting: Performing well on training data but poorly on unseen data.
17. Embedding: A dense numerical representation of a word or item.
18. DenseNet: A CNN architecture with dense connections that promote feature reuse.
19. Domain adaptation: Adapting a model to a different data domain.
20. Object detection: Identifying objects and their locations within an image.

How to answer practical questions tomorrow

For every practical, follow this order when speaking:

1. State the aim.
2. Explain the dataset.
3. Name the important libraries.
4. Explain the model architecture.
5. Describe the training process.
6. Explain the output and evaluation.
7. Give one real-world application.

If the examiner asks about a function, answer in one sentence: what it does and why your program needs it.

One final point: For Practical 6, be precise: the provided VGG16 program is an image-classification implementation, not full object detection with bounding boxes.

unit wise give me topic in details in q format

Deep Learning (414443) — Unit-wise Detailed Viva Questions and Answers

SPPU Final Year IT, 2019 Course

I'll organize the syllabus into six units, with questions in exam/viva format and detailed answers in simple English. Each answer will explain the definition, working, important points, examples, and advantages or applications wherever relevant.

Unit I — Fundamentals of Deep Learning

Q1. What is Deep Learning? Explain its features and applications.

Answer:

Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns from large amounts of data.

Traditional programming requires rules to be written explicitly. In deep learning, a model learns patterns from training examples and uses those patterns to make predictions.

Features of deep learning:

1. Uses neural networks with multiple layers.
2. Learns features automatically from data.
3. Can process images, audio, text, and numerical data.
4. Often benefits from large datasets and powerful computing resources.
5. Can solve complex classification and prediction problems.

Applications:

- Image and face recognition
- Speech recognition
- Natural language processing
- Medical image analysis
- Fraud detection
- Recommendation systems
- Autonomous vehicles

Example: A deep learning model can learn from thousands of handwritten digit images and then predict which digit appears in a new image.

Q2. Explain an Artificial Neural Network (ANN) and its architecture.

Answer:

An Artificial Neural Network is a computing model inspired by the information-processing structure of biological neurons. It consists of interconnected artificial neurons arranged in layers.

The main components are:

1. Input layer: Receives the input features.
2. Hidden layer(s): Processes inputs and learns patterns.
3. Output layer: Produces the final prediction.
4. Weights: Control the strength of connections between neurons.
5. Bias: Adds an adjustable value to a neuron's weighted input.
6. Activation function: Introduces non-linearity.

A neuron generally calculates:

\[z=\sum_{i=1}^{n}w_i x_i+b \]

It then applies an activation function:

\[a=f(z) \]

Here, \(x_i\) represents input values, \(w_i\) represents weights, \(b\) is bias, and \(f\) is the activation function.

Working:

1. Input data enters the input layer.
2. Each neuron calculates a weighted sum plus bias.
3. The activation function transforms the result.
4. Information moves through the hidden layers.
5. The output layer generates a prediction.
6. During training, weights are adjusted to reduce prediction error.

Q3. What is a Multilayer Perceptron (MLP)? Explain its architecture.

Answer:

A Multilayer Perceptron is a type of feedforward neural network that contains an input layer, one or more hidden layers, and an output layer. Neurons in adjacent layers are typically fully connected.

Architecture:

- Input layer: Receives features such as age, height, or image pixel values.
- Hidden layers: Learn combinations of features.

- Output layer: Produces the final result.

Working:

1. Input values are supplied to the network.
2. Each layer calculates weighted sums and applies activation functions.
3. The output layer generates the prediction.
4. The loss function measures the prediction error.
5. Backpropagation calculates gradients.
6. An optimizer updates the weights.

Advantages:

- Learns nonlinear relationships.
- Can be used for classification and regression.
- Can learn complex patterns from data.

Limitations:

- Can require substantial training data.
- May overfit.
- Fully connected layers can contain many parameters.

Applications: Digit recognition, tabular-data classification, and prediction tasks.

Q4. Explain the Feedforward Neural Network.

Answer:

A Feedforward Neural Network (FNN) is a neural network in which information moves from the input layer through hidden layers to the output layer, without recurrent feedback connections.

Working:

1. The input layer receives the data.
2. Hidden neurons calculate weighted sums.
3. Activation functions transform the values.
4. Each layer passes its output to the next layer.
5. The output layer generates a prediction.

Example: An FNN trained on the MNIST dataset can classify an image of a handwritten digit from 0 to 9.

Advantages:

- Simple architecture.
- Suitable for many classification tasks.
- Straightforward to implement and train.

Limitations:

- Does not inherently preserve spatial relationships in images.
- Does not inherently maintain sequence memory.
- Large fully connected networks may have many parameters.

Q5. Explain Backpropagation in detail.

Answer:

Backpropagation is an algorithm used to calculate the gradients of the loss function with respect to the weights and biases of a neural network. These gradients help the optimizer update the parameters to reduce prediction error.

Steps:

1. Initialize parameters: Assign initial weights and biases.
2. Forward propagation: Pass input data through the network.
3. Calculate loss: Compare the predicted output with the actual output.
4. Calculate gradients: Propagate error-related derivatives backward through the network using the chain rule.
5. Update parameters: Use an optimizer, such as gradient descent, to change the weights and biases.
6. Repeat: Continue over batches and epochs until training finishes.

A basic gradient-descent update is:

$$ w_{\text{new}}=w_{\text{old}}-\eta\frac{\partial L}{\partial w} $$

Where:

- w = weight
- L = loss function
- η = learning rate

Advantages:

- Efficiently computes gradients in multilayer networks.

- Makes neural-network training possible.
- Works with many architectures, including MLPs and CNNs.

Important distinction: Backpropagation calculates gradients; the optimizer uses those gradients to update parameters.

Q6. What is Gradient Descent? Explain its types.

Answer:

Gradient Descent is an optimization algorithm used to minimize the loss function by updating model parameters in the direction opposite to the gradient.

$$\theta_{\text{new}}=\theta_{\text{old}}-\eta\nabla_\theta L$$

Types:

1. Batch Gradient Descent: Uses the entire training dataset to calculate each update.
2. Stochastic Gradient Descent (SGD): Updates parameters using one training example at a time.
3. Mini-batch Gradient Descent: Uses a small group of training examples for each update.

Type	Data per update	Main characteristic
Batch	Entire dataset	Stable but can be expensive
Stochastic	One example	Frequent, noisy updates
Mini-batch	Small batch	Balances efficiency and update stability

Learning rate: A large learning rate may cause unstable updates. A very small learning rate may make training slow.

Q7. Explain the vanishing gradient problem.

Answer:

The vanishing gradient problem occurs when gradients become extremely small as they propagate backward through many layers. Earlier layers then receive very small updates and learn slowly.

Causes:

- Repeated multiplication of small derivatives.
- Deep networks with certain activation functions.
- Long sequences in some recurrent neural networks.

Effects:

1. Earlier layers learn slowly.
2. Training may take longer.
3. The network may struggle to learn long-range dependencies.

Ways to reduce it:

- Use ReLU-family activation functions where appropriate.
- Use suitable weight initialization.
- Use normalization techniques where appropriate.
- Use architectures such as LSTM for sequence-learning problems.
- Use residual or skip connections in suitable deep architectures.

Q8. Explain ReLU, Leaky ReLU and ELU activation functions.

Answer:

Activation functions introduce non-linearity into neural networks, allowing them to learn more complex patterns.

1. ReLU

ReLU stands for Rectified Linear Unit.

$$f(x)=\max(0,x)$$

- If x is positive, the output is x.
- If x is negative, the output is zero.

Advantages: Simple, computationally efficient, and commonly used in hidden layers.

Limitation: Some neurons can stop updating when their outputs remain zero, known as the dying ReLU problem.

2. Leaky ReLU

Leaky ReLU allows a small negative output for negative inputs.

$$f(x)= \begin{cases} x & x>0\\ \alpha x & x\leq0 \end{cases}$$

Here, α is a small positive constant.

Advantage: Allows a nonzero gradient on the negative side.

3. ELU

ELU stands for Exponential Linear Unit.

$$f(x)= \begin{cases} x & x>0\\ \alpha(e^x-1) & x\leq0 \end{cases}$$

ELU produces smooth negative outputs and linear positive outputs.

Comparison: ReLU is simple and fast; Leaky ReLU allows a small negative slope; ELU uses a smooth exponential curve for negative inputs.

Q9. What is an optimization algorithm? Explain SGD and Adam.

Answer:

An optimization algorithm updates model parameters to minimize the loss function.

Stochastic Gradient Descent (SGD):

- Uses gradients to update parameters.
- Is relatively simple.
- Can be sensitive to learning-rate selection.
- May require careful tuning for efficient convergence.

Adam:

Adam stands for Adaptive Moment Estimation.

- Maintains moving averages of gradients.
- Maintains moving averages of squared gradients.
- Adapts the effective learning rate for different parameters.
- Is commonly used in deep learning.

Difference: SGD generally uses a common learning-rate scale, while Adam adapts updates using estimates based on gradient history.

Q10. Explain hyperparameters in deep learning.

Answer:

Hyperparameters are settings selected before or during model configuration that control how training takes place or how the model is structured.

Examples:

- Learning rate
- Batch size
- Number of epochs
- Number of hidden layers
- Number of neurons per layer
- Dropout rate
- Optimizer choice

Importance: Poor hyperparameter choices can make training slow, unstable, or inaccurate.

Hyperparameters can be selected using manual experimentation, grid search, random search, or other tuning techniques.

Q11. What is regularization? Explain its techniques.

Answer:

Regularization refers to techniques used to reduce overfitting and improve a model's performance on unseen data.

Common techniques:

1. L1 regularization: Adds a penalty based on the absolute values of weights. It can encourage some weights to become zero.
2. L2 regularization: Adds a penalty based on squared weights. It discourages excessively large weights.
3. Dropout: Randomly disables some units during training.
4. Early stopping: Stops training when validation performance stops improving.
5. Data augmentation: Creates modified training examples, especially for images.

Overfitting example: A model obtains very high training accuracy but low test accuracy because it has learned training-specific details that do not generalize.

Unit II — Convolutional Neural Networks (CNN)

Q1. What is a Convolutional Neural Network? Explain its architecture.

Answer:

A Convolutional Neural Network (CNN) is a deep learning architecture mainly used to process images and other grid-like data. It automatically learns features such as edges, textures, shapes, and objects.

Architecture of CNN

1. Input layer: Receives the image as pixel values.
2. Convolution layer: Applies filters to extract features.
3. Activation layer: Usually uses ReLU to introduce non-linearity.
4. Pooling layer: Reduces the spatial dimensions of feature maps.
5. Flatten layer: Converts feature maps into a one-dimensional vector.
6. Fully connected layer: Uses the extracted features for classification.
7. Output layer: Produces class scores or probabilities.

Input Image

Convolution + ReLU

Pooling

Flatten + Fully Connected Layer

Output Class

Applications: Image classification, face recognition, medical imaging, object detection, and image segmentation.

Q2. Explain the convolution operation with an example.

Answer:

Convolution is an operation in which a small matrix called a filter or kernel moves across an input image and calculates values from local regions.

The purpose is to identify useful features, such as edges and patterns.

Suppose an image is represented by a matrix. A \(3\times3\) filter slides over the image. At each position:

1. The filter overlaps a small image region.
2. Corresponding values are multiplied.
3. The products are added together.
4. The result is stored in the output feature map.
5. The filter moves to the next position.

The process continues until the filter has covered the input.

Example: A filter designed to detect vertical edges produces stronger responses where vertical edges appear in an image.

Important terms:

- Kernel: The small matrix used to extract features.
- Stride: How far the kernel moves at each step.
- Padding: Extra pixels added around the input.
- Feature map: The output generated by the convolution operation.

Q3. What is a filter or kernel in CNN?

Answer:

A filter, also called a kernel, is a small matrix of learnable weights used to extract features from an input image.

Different filters can learn to recognize different patterns.

Examples include:

- Edges
- Corners
- Textures
- Shapes
- Complex object features

During training, the network adjusts filter weights using backpropagation and an optimizer.

Advantages:

1. Automatically learns useful features.
2. Reuses the same weights at different image positions.
3. Reduces the number of parameters compared with many fully connected alternatives.

Q4. Explain parameter sharing in CNN.

Answer:

Parameter sharing means that a convolution filter uses the same weights at different positions of the input image.

For example, a filter that learns to detect a vertical edge can apply the same learned pattern to the left, middle, or right side of an image.

Advantages:

1. Reduces the number of trainable parameters.
2. Makes CNNs computationally efficient.
3. Allows the same feature to be detected at different locations.
4. Helps CNNs learn reusable visual patterns.

Parameter sharing is one of the major differences between convolutional layers and fully connected layers.

Q5. Explain translation equivariance and translation invariance.

Answer:

These concepts describe how a model responds when an object moves within an image.

Translation equivariance: If an input shifts, the corresponding feature representation generally shifts in a related way.

For example, if a vertical edge moves to the right, its detected feature may also move to the right in the feature map.

Translation invariance: The output remains approximately unchanged when the input undergoes a specified transformation.

For example, an image classifier may predict “cat” whether the cat appears near the left or right side of the image.

Difference:

Translation equivariance	**Translation invariance**
The representation changes in a corresponding way to the input shift.	The final representation or prediction remains approximately unchanged.
Commonly associated with convolution operations.	Can be encouraged by pooling, augmentation, and suitable architectures.
Retains information about where features occur.	Reduces sensitivity to location changes.

A standard CNN is not automatically perfectly translation invariant.

Q6. Explain pooling in CNN. Describe its types.

Answer:

Pooling is a downsampling operation that reduces the spatial dimensions of feature maps. It can reduce computation and help the network become less sensitive to small shifts.

Types of pooling

1. Max Pooling

Selects the maximum value from each local region.

Example:

Input region:

$$\begin{bmatrix} 1&3\\ 2&8 \end{bmatrix}$$

Max pooling produces:

$$[8]$$

2. Average Pooling

Calculates the average of values in each local region.

For the same input:

$$\frac{1+3+2+8}{4}=3.5$$

Advantages of pooling:

- Reduces spatial dimensions.
- Can reduce computation.
- Retains important local information.
- Can improve robustness to small positional changes.

Limitation: Pooling can discard useful spatial information.

Q7. Explain stride and padding in CNN.

Answer:

Stride is the number of positions by which a filter moves each time.

- Stride 1: The filter moves one position at a time.
- Stride 2: The filter moves two positions at a time, generally producing a smaller feature map.

Padding adds pixels around the boundary of the input, often using zeros.

Two common padding types are:

- Valid: No padding is added.
- Same: Padding is chosen to preserve the spatial dimensions for stride 1.

For a one-dimensional input dimension N, filter size F, padding P, and stride S, the output dimension is:

$$ O=\left\lfloor\frac{N+2P-F}{S}\right\rfloor+1 $$

For two-dimensional images, this calculation is applied separately to height and width.

Q8. What are the different types or variants of CNN?

Answer:

CNN architectures have evolved to improve accuracy, computation, feature reuse, and the ability to process different tasks.

Some important examples are:

1. LeNet: An early CNN architecture used for handwritten digit recognition.
2. AlexNet: A deep CNN that achieved major success in ImageNet classification in 2012.
3. VGG: Uses a simple architecture with stacks of small convolution filters.
4. GoogLeNet/Inception: Uses parallel convolution operations of different sizes within Inception modules.

5. ResNet: Uses residual or skip connections to support the training of very deep networks.
6. DenseNet: Connects each layer to earlier layers within dense blocks to encourage feature reuse.

Different architectures make different trade-offs between accuracy, model size, training cost, and inference speed.

Q9. Explain AlexNet architecture and its importance.

Answer:

AlexNet is a deep CNN architecture that became famous after winning the ImageNet Large Scale Visual Recognition Challenge in 2012.

Main components:

1. Input image.
2. Convolutional layers for feature extraction.
3. ReLU activation functions.
4. Pooling layers to reduce spatial dimensions.
5. Fully connected layers for classification.
6. Softmax output for class probabilities.

AlexNet also used techniques such as dropout to help reduce overfitting.

Importance:

- Demonstrated the effectiveness of deep CNNs on large image datasets.
- Helped popularize GPU-based deep learning.
- Encouraged further research into deeper and more powerful CNN architectures.

Applications: Image classification, visual recognition, and feature extraction.

Q10. Differentiate between CNN and FNN.

Answer:

CNN	**FNN**
Uses convolutional filters.	Commonly uses fully connected layers.
Preserves spatial structure in its convolutional stages.	Often flattens image inputs into vectors.

Uses parameter sharing.	Fully connected layers generally have separate weights for separate connections.
Efficient at learning local image patterns.	Suitable for many fixed-size feature-based tasks.
Commonly used for image processing.	Commonly used for tabular classification and regression.

Unit III — Recurrent Neural Networks (RNN)

Q1. What is a Recurrent Neural Network? Explain its architecture.

Answer:

A Recurrent Neural Network is a type of neural network designed to process sequential data. It maintains a hidden state that carries information from previous steps to later steps.

RNNs are useful when the order of data matters.

Examples of sequential data:

- Sentences
- Speech signals
- Stock-price time series
- Weather measurements
- Sensor readings

Architecture and working

At each time step, the RNN receives the current input and the previous hidden state.

A simplified equation is:

\[h_t=\tanh(W_xx_t+W_hh_{t-1}+b) \]

Where:

- \(x_t\) = current input
- \(h_{t-1}\) = previous hidden state
- \(h_t\) = new hidden state
- \(W_x,W_h\) = learned weight matrices
- \(b\) = bias

The model can then use the hidden state to produce an output.

Working:

1. Receive the current sequence element.
2. Combine it with information from the previous hidden state.
3. Calculate the new hidden state.
4. Generate an output if required.
5. Repeat for the next sequence element.

Q2. Explain the types of RNN architectures.

Answer:

RNN architectures can be classified according to the relationship between input and output sequences.

1. One-to-One

One input produces one output.

Example: A basic fixed-size classification task.

2. One-to-Many

One input produces a sequence of outputs.

Example: Image caption generation, where an image is used to generate a sentence.

3. Many-to-One

A sequence of inputs produces one output.

Example: Sentiment classification, where a complete review is classified as positive or negative.

4. Many-to-Many

A sequence of inputs produces a sequence of outputs.

Example: Sequence labelling or machine translation.

Some many-to-many tasks produce outputs at each input time step, while encoder-decoder tasks can generate an output sequence after encoding the input.

Q3. Differentiate between FNN and RNN.

Answer:

FNN	**RNN**
Information flows forward through layers.	Uses recurrent connections across time steps.
Has no built-in recurrent hidden state.	Maintains a hidden state.

Suitable for fixed-size input tasks.	Suitable for sequential data.
Does not inherently preserve earlier sequence information.	Can carry information from earlier sequence steps.
Often used for tabular classification.	Often used for text and time-series tasks.

Q4. What is the vanishing gradient problem in RNN?

Answer:

The vanishing gradient problem occurs when gradients become very small during backpropagation through many time steps.

As a result, the network struggles to learn relationships between events that are far apart in a sequence.

Example: In a long paragraph, the meaning of a word near the end may depend on a word appearing much earlier. A basic RNN may struggle to preserve that information.

Effects:

1. Earlier sequence steps receive small updates.
2. Long-term dependencies are difficult to learn.
3. Training may become less effective.

Solutions:

- LSTM
- GRU
- Suitable initialization and optimization
- Gradient clipping to control exploding gradients
- Attention mechanisms or other sequence architectures

Gradient clipping mainly addresses exploding gradients rather than directly solving vanishing gradients.

Q5. What is LSTM? Explain its architecture.

Answer:

LSTM stands for Long Short-Term Memory. It is a special type of RNN designed to learn short-term and long-term dependencies more effectively than a basic RNN.

An LSTM uses a cell state and gates to regulate information flow.

Main components

1. Forget gate: Determines what information from the previous cell state should be discarded.

2. Input gate: Determines which new information should be stored.

3. Cell state: Carries information through the sequence.

4. Output gate: Determines what information is used to produce the hidden state.

Working

1. The forget gate evaluates the previous cell state and current input.
2. The input gate selects new information to store.
3. The cell state is updated.
4. The output gate produces the new hidden state.
5. The process repeats for the next time step.

Advantages:

- Can learn longer dependencies.
- Helps reduce the vanishing-gradient problem.
- Useful for variable-length sequential data.

Applications: Speech recognition, language modelling, time-series forecasting, and sequence classification.

Q6. Differentiate between RNN and LSTM.

Answer:

RNN	**LSTM**
Has a simpler recurrent structure.	Has a cell state and gates.
Can struggle with long-term dependencies.	Designed to handle longer dependencies more effectively.
Usually has fewer parameters.	Usually has more parameters.
May experience severe vanishing gradients.	Helps mitigate vanishing gradients.
Can be suitable for simpler sequence tasks.	Often useful for more complex sequence tasks.

Q7. Explain the encoder-decoder architecture.

Answer:

An encoder-decoder architecture converts an input into an internal representation and then uses that representation to produce an output.

It is commonly used for tasks where the input and output sequences differ in length.

Components

Encoder: Processes the input sequence and creates a representation.

Decoder: Uses the representation to generate the output sequence.

Working

1. The encoder reads the input.
2. It creates an internal representation of the input information.
3. The decoder receives this representation.
4. The decoder generates output elements one by one or in a suitable sequence-processing manner.

Example: In machine translation, an encoder processes an English sentence and a decoder generates its translation into another language.

Applications:

- Machine translation
- Text summarization
- Conversational systems
- Sequence generation

Q8. What is a Recursive Neural Network? How is it different from an RNN?

Answer:

A Recursive Neural Network applies neural computations over hierarchical structures, such as trees. It combines information from child nodes to create representations of parent nodes.

For example, in sentence analysis, a model may combine word representations into phrase representations and then into a sentence representation.

Difference:

Recurrent Neural Network	**Recursive Neural Network**
Processes a sequence over time steps.	Processes hierarchical or tree-structured data.
Passes hidden states across sequence positions.	Combines representations according to a recursive structure.
Useful for text sequences and time series.	Useful for tree-structured language representations.

The terms sound similar but describe different structures.

Q9. Explain the applications of RNN.

Answer:

RNNs are used in tasks where the order of information matters.

1. Speech recognition: Processes audio features over time.
2. Language modelling: Predicts likely next tokens.
3. Sentiment analysis: Uses word order and context to classify text.
4. Time-series prediction: Learns patterns in historical measurements.
5. Machine translation: Processes source sequences and contributes to generating target sequences.
6. Sequence labelling: Assigns labels to sequence elements.

Unit IV — Autoencoders

Q1. What is an Autoencoder? Explain its architecture and working.

Answer:

An autoencoder is a neural network that learns to reconstruct its input. It typically contains an encoder, a latent representation, and a decoder.

Architecture

1. Encoder: Converts the input into a compact or otherwise constrained representation.

2. Latent space: Contains the internal representation learned by the encoder.

3. Decoder: Reconstructs the input from the latent representation.

Input x

Encoder

Latent z

Reconstruction $\hat{x}$

Decoder

Working

1. Input data is provided to the encoder.
2. The encoder learns a representation.
3. The decoder reconstructs the input.
4. The reconstruction is compared with the original.

5. The model is trained to reduce reconstruction loss.

A common reconstruction loss is Mean Squared Error:

$$ L=\frac{1}{n}\sum_{i=1}^{n}(x_i-\hat{x}_i)^2 $$

Applications: Data compression, feature learning, denoising, and anomaly detection.

Q2. What is an undercomplete autoencoder?

Answer:

An undercomplete autoencoder has a latent representation with fewer dimensions than the original input.

For example, a 100-dimensional input may be compressed into a 20-dimensional latent representation.

Working:

1. The encoder reduces the dimensionality.
2. The latent representation stores important information.
3. The decoder reconstructs the input.
4. The training objective encourages the model to retain useful information despite the reduced representation.

Advantages:

- Learns compact representations.
- Can perform dimensionality reduction.
- May remove some redundant information.

Limitation: Important information can be lost if the latent representation is too small.

Q3. Explain a regularized autoencoder.

Answer:

A regularized autoencoder uses additional constraints or penalties during training to encourage useful representations instead of simply memorizing the input.

Common approaches include:

- Sparse autoencoders
- Denoising autoencoders
- Contractive autoencoders

Regularization may encourage sparse activation, robustness to noise, or reduced sensitivity to small changes in the input.

Purpose: To improve representation learning and generalization.

Q4. What is a Sparse Autoencoder?

Answer:

A sparse autoencoder encourages only a small number of hidden units to activate for a given input, even when the latent layer contains many units.

This is commonly achieved by adding a sparsity penalty to the reconstruction loss.

The objective can be represented as:

\[L_{\text{total}}=L_{\text{reconstruction}}+\lambda L_{\text{sparsity}} \]

Here, \(\lambda\) controls the strength of the sparsity penalty.

Advantages:

1. Learns selective features.
2. Can learn useful representations without requiring a very small latent layer.
3. Encourages efficient representation of data.

Applications: Feature extraction and unsupervised representation learning.

Q5. Explain a Denoising Autoencoder.

Answer:

A denoising autoencoder learns to reconstruct clean data from a corrupted version of the input.

Working:

1. Start with clean training data.
2. Add noise or another form of corruption to the input.
3. Provide the corrupted input to the encoder.
4. Decode the latent representation.
5. Compare the reconstructed output with the original clean data.
6. Train the model to reduce reconstruction error.

Example: A denoising autoencoder can learn to remove random noise from an image.

Advantages:

- Learns representations that are robust to some corruption.
- Can be used for noise removal.
- Can improve feature learning.

Q6. What is a Contractive Autoencoder?

Answer:

A contractive autoencoder is designed to make its learned representation less sensitive to small changes in the input.

It typically adds a penalty based on how sensitive the encoder's representation is to changes in input values. This often involves the encoder's Jacobian.

Working:

1. Encode the input.
2. Reconstruct the input.
3. Calculate reconstruction loss.
4. Add a penalty for excessive sensitivity of the representation.
5. Train the model to minimize the combined objective.

Purpose: To learn representations that remain relatively stable under small input variations.

Q7. Explain stochastic autoencoders.

Answer:

The term stochastic autoencoder can describe autoencoder approaches that introduce randomness into the encoding or latent representation. The exact definition depends on the formulation used in the course material.

In a common probabilistic approach, the encoder produces parameters of a probability distribution rather than only a fixed latent vector. A latent representation is then sampled from that distribution.

A well-known related architecture is the Variational Autoencoder (VAE).

Applications of stochastic latent representations:

- Generative modelling
- Learning probabilistic representations
- Data generation
- Representation learning

Note: If your lecturer has defined a specific stochastic autoencoder formulation in the notes, use that exact definition in the oral exam.

Q8. What is anomaly detection using an autoencoder?

Answer:

Anomaly detection identifies data points that differ from normal patterns.

An autoencoder can be trained primarily on normal examples so that it learns to reconstruct normal data effectively.

Working

1. Collect training examples representing normal behaviour.
2. Train an autoencoder to reconstruct those examples.
3. Pass new examples through the trained model.
4. Calculate the reconstruction error.
5. Compare the error with a selected threshold.
6. Flag unusually high errors for further investigation.

Example: In credit-card transaction analysis, a transaction with an unusually high reconstruction error may be flagged for investigation.

Advantages:

- Can help identify previously unseen anomalies.
- Can work when labelled anomalies are scarce.
- Learns patterns from normal data.

Limitations:

- A high error does not prove fraud.
- The threshold affects false positives and false negatives.
- An autoencoder may also reconstruct some anomalous examples well.

Q9. Explain the applications of autoencoders.

Answer:

1. Dimensionality reduction: Learns a compact representation.
2. Denoising: Removes some forms of noise from data.
3. Anomaly detection: Identifies unusual examples using reconstruction error.
4. Feature extraction: Learns representations for downstream tasks.
5. Data compression: Stores data in a lower-dimensional representation.
6. Generative modelling: Probabilistic variants such as VAEs can generate new samples.

Q10. Differentiate between PCA and Autoencoder.

Answer:

PCA	Autoencoder
A linear dimensionality-reduction method.	A neural-network-based representation-learning method.
Finds directions that explain variance.	Learns representations by optimizing a training objective.
Standard PCA is based on linear transformations.	Can learn nonlinear representations with nonlinear layers.
Usually simpler and faster for many small or medium datasets.	May require more computation and training.
Has a direct mathematical solution or efficient numerical methods.	Generally learns parameters through optimization.

Unit V — Representation Learning

Q1. What is Representation Learning? Explain its importance.

Answer:

Representation learning is a machine learning approach in which a model learns useful features or representations from raw data rather than relying entirely on manually designed features.

For example, when processing an image, a neural network may learn edges in early layers, textures and shapes in intermediate layers, and more complex patterns in later layers.

Importance:

1. Reduces the need for manual feature engineering.
2. Helps models discover useful patterns.
3. Can improve performance on complex tasks.
4. Supports transfer learning.
5. Can produce features suitable for classification, clustering, and prediction.

Applications: Computer vision, NLP, speech recognition, and recommendation systems.

Q2. Explain Greedy Layer-wise Pretraining.

Answer:

Greedy layer-wise pretraining is a training approach in which individual layers or groups of layers are pretrained one at a time before training the complete network together.

The word greedy means that each layer is trained locally at its stage rather than optimizing all layers jointly from the beginning.

Working

1. Train the first layer to learn a useful representation of the input.
2. Use the representation from the first layer as input for the next layer.
3. Train the next layer.
4. Continue for the remaining layers.
5. Stack the pretrained layers.
6. Optionally fine-tune the complete network using a task-specific objective.

Advantages:

- Historically helped initialize deep networks.
- Can encourage useful intermediate representations.
- Can provide a structured way to train layers.

Limitation: It adds complexity and is not always necessary with modern initialization and end-to-end training techniques.

Q3. What is Transfer Learning? Explain its working and advantages.

Answer:

Transfer learning reuses knowledge learned by a model on one task for another related task.

For example, a CNN trained on a large image dataset may learn general features such as edges, textures, and shapes. These features can be reused for a smaller medical-image or plant-classification dataset.

Working

1. Select a suitable pretrained model.
2. Load its pretrained weights.
3. Remove or replace the original task-specific output layers when needed.
4. Freeze the pretrained layers initially, if appropriate.
5. Add new layers for the target task.
6. Train the new layers.
7. Optionally unfreeze selected pretrained layers and fine-tune them.

Advantages:

- Reduces training time.
- Can work with smaller target datasets.

- Reuses previously learned features.
- May improve performance when the source and target tasks are related.

Applications: Image classification, speech processing, and language models.

Q4. What is Domain Adaptation?

Answer:

Domain adaptation is a machine learning approach that adapts a model trained on a source domain so that it performs well on a target domain with a different data distribution.

Example: A model trained to classify photographs taken in daylight may perform poorly on nighttime images. Domain adaptation attempts to reduce this performance gap.

Working

1. Train or obtain a model using source-domain data.
2. Identify differences between the source and target domains.
3. Apply suitable adaptation methods.
4. Evaluate the model on the target domain.

Advantages:

- Helps handle changes in data distributions.
- Can reduce the need for large amounts of target-domain labels.
- Can improve real-world generalization.

Q5. Differentiate between Transfer Learning and Domain Adaptation.

Answer:

Transfer Learning	**Domain Adaptation**
Broadly reuses knowledge from a source task or model.	Specifically addresses differences between source and target domains.
Can involve different but related tasks.	Often focuses on a related task with different data distributions.
May use pretrained weights and fine-tuning.	May use feature alignment, reweighting, or other adaptation methods.
Example: Reusing an image model for a new classification task.	Example: Adapting an image classifier from daylight images to nighttime images.

Domain adaptation is commonly treated as a specialized form of transfer learning.

Q6. What is Distributed Representation?

Answer:

A distributed representation represents an input or concept through a pattern of activations across multiple units rather than assigning one dedicated unit to each concept.

For example, a word embedding represents a word using a vector of numerical values. Different combinations of values can represent different words and relationships.

Advantages:

1. Can represent many concepts using combinations of features.
2. Supports similarity calculations between representations.
3. Can capture useful relationships in data.
4. Supports feature sharing between related concepts.

Applications: Word embeddings, image representations, and learned features in neural networks.

Q7. Explain DenseNet architecture.

Answer:

DenseNet stands for Densely Connected Convolutional Network. It is a CNN architecture in which each layer within a dense block receives feature maps from earlier layers in that block.

This design encourages feature reuse and provides multiple paths for information and gradients to flow through the network.

Working

1. An initial convolution processes the input.
2. A layer generates new feature maps.
3. Those feature maps are combined with feature maps from preceding layers.
4. Later layers receive information from several earlier layers.
5. Transition layers may reduce feature-map dimensions between dense blocks.
6. A classification head produces the final output.

Advantages:

- Encourages feature reuse.
- Improves information flow.
- Can help gradient propagation.
- Can achieve parameter efficiency relative to some comparable architectures.

Limitations:

- Concatenating many feature maps can increase memory use.
- Dense connections can increase implementation and computation complexity.

Q8. Differentiate between ResNet and DenseNet.

Answer:

ResNet	DenseNet
Uses residual or skip connections.	Uses dense connections within dense blocks.
Adds a shortcut input to a transformed output.	Concatenates earlier feature maps with new feature maps.
Supports information and gradient flow through shortcuts.	Encourages feature reuse across layers.
Commonly uses addition for residual merging.	Commonly uses concatenation for dense connectivity.

Q9. What is feature extraction in deep learning?

Answer:

Feature extraction is the process of transforming raw data into useful representations that capture important patterns.

In deep learning, the network learns features during training.

Example: In an image model:

- Early layers may learn edges.
- Intermediate layers may learn textures and shapes.
- Later layers may learn complex visual patterns.

Pretrained models can be used as feature extractors by keeping their learned layers fixed and training a new classifier on top.

Q10. What are the applications of representation learning?

Answer:

1. Image recognition: Learns visual features.
2. Natural language processing: Learns word and sentence representations.
3. Speech recognition: Learns representations from audio.
4. Recommendation systems: Learns user and item representations.

5. Clustering: Learns features that may make similar data points easier to group.
6. Transfer learning: Reuses learned representations in related tasks.

Unit VI — Applications of Deep Learning

Q1. Explain Image Classification using Deep Learning.

Answer:

Image classification is the process of assigning a class label to an image.

Examples include identifying whether an image contains a cat, dog, car, or person.

Working

1. Collect labelled images.
2. Resize and preprocess the images.
3. Divide the data into training, validation, and test sets as appropriate.
4. Build a CNN or use a pretrained model.
5. Train the model using image labels.
6. Evaluate it on unseen images.
7. Use the trained model to predict classes for new images.

Common architectures: CNN, VGG16, ResNet, and DenseNet.

Applications:

- Medical image classification
- Plant disease identification
- Handwritten digit recognition
- Product categorization
- Quality inspection

Q2. Differentiate between Image Classification and Object Detection.

Answer:

Image Classification	**Object Detection**
Predicts the class of an image.	Predicts object classes and their locations.
May produce one label for the entire image.	Can identify multiple objects in one image.

Does not inherently require bounding boxes.	Usually predicts bounding boxes and class scores.
Example: Identifying an image as containing a dog.	Example: Locating three dogs in an image.

Q3. Explain the application of deep learning in Social Network Analysis.

Answer:

Social Network Analysis studies relationships and interactions between people, groups, accounts, or other entities.

A social network can be represented as a graph:

- Nodes: People, accounts, or organizations.
- Edges: Relationships or interactions between nodes.

Deep learning can learn patterns from node features, edges, and interaction histories.

Applications:

1. Community detection: Identifying groups with strong connections.
2. Link prediction: Predicting whether two users may connect.
3. Recommendation: Suggesting people, groups, or content.
4. Fraud detection: Identifying suspicious account or interaction patterns.
5. Content analysis: Classifying or analysing posts.

Graph neural networks are one family of models used for learning from graph-structured data.

Q4. Explain Speech Recognition using Deep Learning.

Answer:

Speech recognition converts spoken language into text or another machine-readable representation.

Working

1. Capture speech using a microphone.
2. Convert the audio into a digital signal.
3. Extract or learn useful audio representations.
4. Process the audio with a suitable neural network.
5. Predict the sequence of spoken words.

6. Produce the transcription.

Modern systems may use convolutional networks, recurrent networks, attention-based architectures, or Transformer models.

Applications:

- Voice assistants
- Automatic transcription
- Voice-controlled systems
- Meeting transcription
- Accessibility tools

Challenges:

- Background noise
- Different accents
- Fast speech
- Similar-sounding words
- Multiple speakers

Q5. Explain Deep Learning in Recommender Systems.

Answer:

A recommender system predicts which items a user may prefer and suggests relevant content or products.

Examples include recommending movies, songs, products, and online courses.

Working

1. Collect user-item interactions, ratings, or other relevant features.
2. Process the data.
3. Learn representations of users and items.
4. Estimate user preferences or item relevance.
5. Rank the candidate items.
6. Recommend the most relevant items.

Deep learning may use embeddings, neural collaborative filtering, sequence models, or other architectures.

Advantages:

- Can learn complex user-item relationships.
- Can incorporate several types of information.
- Can support personalized ranking.

Challenges:

- Sparse interaction data
- New users or items, known as the cold-start problem
- Privacy considerations
- Recommendation bias

Q6. Explain the application of Deep Learning in Natural Language Processing (NLP).

Answer:

Natural Language Processing enables computers to process, analyse, and generate human language. Deep learning can learn language representations from large collections of text.

Working in a typical text-classification task:

1. Collect text data and labels.
2. Clean and tokenize the text.
3. Convert tokens into numerical representations.
4. Process the representations through a neural network.
5. Calculate the loss and train the model.
6. Predict labels for new text.

Applications:

- Sentiment analysis
- Machine translation
- Text summarization
- Question answering
- Chatbots
- Text classification
- Information extraction

Common model types: Word2Vec embeddings, RNNs, LSTMs, and Transformer-based models.

Q7. What is Sentiment Analysis?

Answer:

Sentiment analysis is an NLP task that identifies the sentiment expressed in text.

The output may classify text as positive, negative, or neutral, depending on the task.

Example:

- “This product is excellent.” — Positive
- “The service was terrible.” — Negative
- “The package arrived today.” — Potentially neutral

Working:

1. Collect labelled text.
2. Tokenize and preprocess it.
3. Convert the text into numerical representations.
4. Train a classification model.
5. Predict sentiment for new text.

Applications: Product reviews, customer feedback, social media analysis, and brand monitoring.

Q8. Explain the use of Deep Learning in Medical Applications.

Answer:

Deep learning can assist medical professionals by analysing images, signals, and clinical data.

Applications:

1. Medical image classification.
2. Detection of suspicious regions in scans.
3. Analysis of medical signals.
4. Supporting risk prediction.
5. Processing medical text and reports.

Advantages:

- Can identify complex patterns.
- Can support analysis of large datasets.
- Can assist with prioritization and decision support.

Limitations: Performance depends on data quality, validation, generalization, and appropriate clinical oversight. A model's prediction should not automatically be treated as a definitive medical diagnosis.

Q9. Explain the difference between Classification and Regression.

Answer:

Classification	Regression
Predicts a category or class.	Predicts a numerical value.
Example: Fraud or non-fraud.	Example: Predicting house prices.
Often uses cross-entropy loss.	Often uses mean squared error or mean absolute error.
Common metrics include accuracy, precision, recall, and F1-score.	Common metrics include MAE, MSE, and RMSE.

Q10. What are the challenges of Deep Learning?

Answer:

1. Large data requirements: Some tasks require substantial training data.
2. High computation cost: Training large models can require GPUs or other accelerators.
3. Overfitting: A model may fail to generalize to unseen data.
4. Hyperparameter tuning: Good settings may require experimentation.
5. Interpretability: Some models are difficult to explain.
6. Bias in data: Models can learn unwanted biases from training data.
7. Deployment challenges: Models must meet real-world latency, memory, and reliability requirements.
8. Privacy and security: Training and deploying models require appropriate data protection.

Final quick revision: Important questions from all six units

These are useful questions to practise answering aloud.

Practise by reading each question and revealing its short answer when ready.

Unit I

1. Explain backpropagation and gradient descent.

Backpropagation calculates gradients of the loss with respect to model parameters. Gradient descent uses those gradients to update parameters and reduce the loss.

Hide answer

Unit I

2. What is the vanishing gradient problem?

It occurs when gradients become extremely small during backward propagation, causing earlier layers to learn slowly.

Hide answer

Unit II

3. What is parameter sharing in CNN?

The same convolution filter weights are reused at different image positions, reducing the number of parameters.

Hide answer

Unit II

4. Explain Max Pooling.

Max Pooling selects the largest value in each local region of a feature map, reducing its spatial dimensions.

Hide answer

Unit III

5. What is LSTM?

LSTM is a type of RNN that uses a cell state and gates to learn long-term dependencies more effectively.

Hide answer

Unit III

6. What is an encoder-decoder model?

The encoder creates a representation of the input, and the decoder uses it to generate an output, such as a translated sentence.

Hide answer

Unit IV

7. How does an autoencoder detect anomalies?

It learns to reconstruct normal data. An unusually high reconstruction error can indicate a potential anomaly.

Hide answer

Unit IV

8. Differentiate between sparse and denoising autoencoders.

A sparse autoencoder encourages few active units. A denoising autoencoder reconstructs clean data from corrupted input.

Hide answer

Unit V

9. What is transfer learning?

It reuses knowledge learned by a pretrained model for a new related task.

Hide answer

Unit V

10. What is DenseNet?

DenseNet connects each layer to earlier layers within dense blocks, encouraging feature reuse and information flow.

Hide answer

Unit VI

11. Differentiate between image classification and object detection.

Classification predicts an image class, while detection predicts object classes and their locations.

Hide answer

Unit VI

12. How is deep learning used in NLP?

It learns representations of text for tasks such as translation, sentiment analysis, summarization, and question answering.

Hide answer

Reveal all answers

Exam tip: For a 6–7 mark answer, write the definition, architecture or working steps, a diagram where relevant, advantages or applications, and a short conclusion. For oral questions, start with a one-sentence definition and then explain the working with a simple example.